

CSC 8830: Computer Vision - Assignment 6

Optical Flow, Motion Tracking, and Planar Multi-view Geometry

Harshit Jain | Panther ID: 002916196

GitHub repository: ADD ACCESSIBLE LINK BEFORE SUBMISSION

1. Purpose and reproducible execution

Two videos are processed with dense Farneback optical flow [1,2]. Consecutive-frame feature tracking uses pyramidal Lucas-Kanade [1,3]. Four photographs of a flat book back cover are related by homographies [4]. All methods use classical image processing, with Python, OpenCV, and NumPy. Matplotlib builds this report.

From the project folder on macOS:

```
python3 -m venv venv
source venv/bin/activate
python -m pip install -r requirements.txt
python optical_flow.py videos/video1.mp4
python optical_flow.py videos/video2.mp4
python tracking_validation.py videos/video1.mp4
python tracking_validation.py videos/video2.mp4
python sfm_planar.py
python report_assets/build_report.py
```

Input video measurements

Video	FPS	Frames	Duration (s)	Size (w x h)
video1	59.364656	1941	32.696	848 x 478
video2	30.111545	2236	74.257	848 x 478

Both videos exceed 30 seconds. Optical flow processes the complete clips; this report analyzes samples within their first 30 seconds. Video length alone does not prove continuous object motion. The selected tracking pairs are dominated by stationary scene features, an important limitation discussed below.

Outputs: results/*_optical_flow.mp4, three flow PNGs per video, frames/*_frame_a.png and *_frame_b.png, tracking CSVs and visualizations, and the planar geometry CSVs and PNGs. The four original book PNGs are included in sfm_images/. The recorded working demonstration must be submitted separately with this PDF.

2. Optical flow: method and evidence

Farneback estimates a dense two-component displacement field using local polynomial image models [2]. Each frame is converted to grayscale and compared with its predecessor. Direction is $\text{atan2}(dy, dx)$, and magnitude is $\sqrt{dx^2 + dy^2}$. Image coordinates increase rightward in x and downward in y .

The HSV display uses hue for direction and saturation 255. Value uses the 95th percentile of magnitudes above 0.1 pixels/frame, with a minimum scale of 1 pixel/frame, then clips to 0-255. Smaller estimates remain black. This suppresses noise and reduces sensitivity to extreme magnitudes. Brightness is relative within a frame and cannot establish absolute speed differences across timestamps or videos.



Figure 1. Video1 at approximately 10 s: original and dense flow. The small floor-level vehicle is the moving object; broad background regions have weak flow.

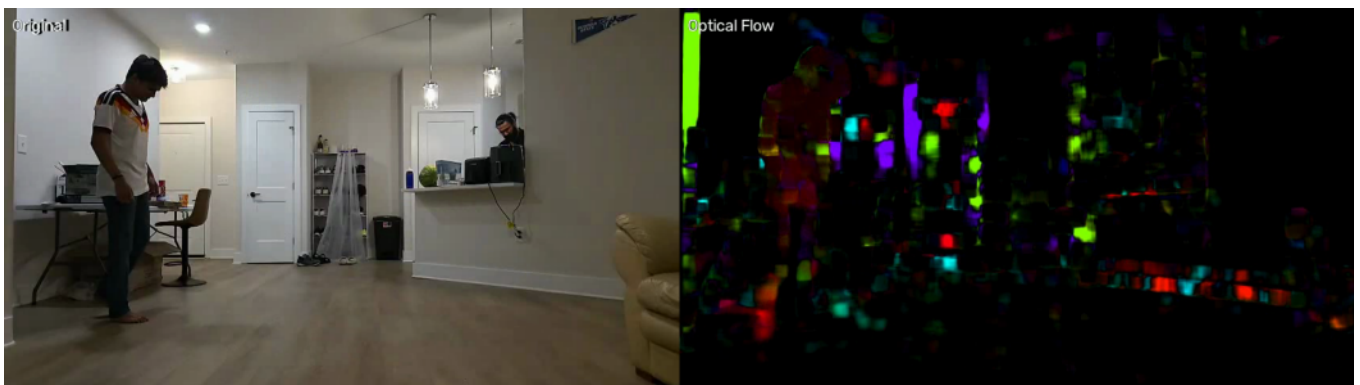


Figure 2. Video2 at approximately 10 s: original and dense flow. Inspect the person and any locally colored regions against the darker background; flow estimates apparent image motion, not physical 3D speed.

In Figure 1, the vehicle region is predominantly red: the implemented hue mapping associates red with rightward motion, yellow-green with downward, cyan with leftward, and violet with upward motion. Figure 2 contains several directions and scattered background estimates, so not every bright patch identifies a moving object. Black means below the display threshold, not proof of zero motion. Camera motion, illumination, occlusion, and compression affect estimates. Mean grayscale difference is a selection heuristic, not physical velocity [1].

3. Tracking equations derived from fundamentals

Brightness constancy assumes the same physical point retains intensity as it moves. Let displacement between the two images be $d=(dx,dy)$ and the time interval be dt [1,3].

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

Expand the right side about (x,y,t) , retaining first-order terms:

$$I(x + \Delta x, y + \Delta y, t + \Delta t) \approx I + I_x \Delta x + I_y \Delta y + I_t \Delta t$$

Cancel I , divide by dt , and define $u=dx/dt$ and $v=dy/dt$:

$$I_x u + I_y v + I_t = 0$$

One pixel gives one equation for two unknowns (the aperture problem). In a local window, assume the same motion for each pixel i . For consecutive-frame displacements, set $b_i=-(I_B(i)-I_A(i))$ and $A_i=[I_x(i), I_y(i)]$. Then $A d$ approximately equals b . Here d has units pixels/frame pair; u and v are pixels/second only if dt is expressed in seconds.

$$E(d) = \|Ad - b\|^2, \quad \nabla_d E = 2A^T(Ad - b) = 0$$

$$(A^T A)d = A^T b, \quad d = (A^T A)^{-1} A^T b$$

The inverse requires a well-conditioned gradient matrix: intensity variation in two directions. This explains selecting corners with `goodFeaturesToTrack`. Iterative refinement warps the second image, estimates a correction, and updates d . Image pyramids reduce large image displacement at coarser scales before refinement [1,3].

Typed worked example of the normal equations

Illustrative gradients (not measured from these videos): A rows are $[1,0]$, $[0,1]$, $[1,1]$; temporal residual $b=[2,1,3]$. Thus $A^T A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ and $A^T b = [5, 4]$. Solving $2dx+dy=5$ and $dx+2dy=4$ gives $dx=2$, $dy=1$ pixels. An initial point $(100,80)$ is predicted at $(102,81)$, with magnitude $\sqrt{5}=2.236$ pixels. Actual video comparisons appear on the next pages.

3.1 Normal equations evaluated on actual video pixels

The theoretical linear system can also be assembled directly from the saved frame pairs, without calling Lucas-Kanade. For point 1 in each video, use a 21x21 window (441 pixels), centered at its initial integer location. Central differences in frame A give $I_x=(I(x+1,y)-I(x-1,y))/2$ and $I_y=(I(x,y+1)-I(x,y-1))/2$. Set $b=-(I_B-I_A)$ and accumulate the normal equations from these measured intensities [1,3].

$$G_{11} = \sum I_x^2, \quad G_{12} = G_{21} = \sum I_x I_y, \quad G_{22} = \sum I_y^2$$

$$c_1 = -\sum I_x(I_B - I_A), \quad c_2 = -\sum I_y(I_B - I_A), \quad Gd = c$$

video1: measured point 1 at (329,141)

$G = [[183857.50, -12931.25], [-12931.25, 282521.25]]$

$c = [-29185.00, -59083.00]$

Solve $Gd=c$: $d=(-0.174006, -0.217092)$ pixels.

Theoretical first-order prediction=(328.825994, 140.782908).

Independent patch location=(329,141); discrepancy=0.278221 pixels.

Pyramidal LK prediction=(328.834076, 140.806015).

video2: measured point 1 at (239,145)

$G = [[421217.75, -51913.25], [-51913.25, 473184.25]]$

$c = [257197.00, -53723.00]$

Solve $Gd=c$: $d=(0.604788, -0.047183)$ pixels.

Theoretical first-order prediction=(239.604788, 144.952817).

Independent patch location=(240,145); discrepancy=0.398018 pixels.

Pyramidal LK prediction=(239.519806, 144.997208).

These are calculations from actual image intensities, not illustrative gradients. The direct first-order solve and OpenCV pyramidal iterative solve need not be identical: their gradients, interpolation, refinement, and scale handling differ. Both are compared with independently matched integer pixel positions. This supplies numerical evidence for the theoretical two-frame tracking formulation while retaining the reference-quantization limitation.

4. Bilinear interpolation: derivation and example

A warped point generally lies between integer pixel centers. Let $x_0 = \text{floor}(x)$, $y_0 = \text{floor}(y)$, $a = x - x_0$, and $b = y - y_0$. Define $I_{00} = I(x_0, y_0)$, $I_{10} = I(x_0 + 1, y_0)$, $I_{01} = I(x_0, y_0 + 1)$, and $I_{11} = I(x_0 + 1, y_0 + 1)$. Bilinear interpolation performs linear interpolation in each direction [5].

$$T = (1 - a)I_{00} + aI_{10}, \quad B = (1 - a)I_{01} + aI_{11}$$

$$I(x, y) = (1 - b)T + bB$$

$$I(x, y) = (1 - a)(1 - b)I_{00} + a(1 - b)I_{10} \\ + (1 - a)bI_{01} + abI_{11}$$

The four weights sum to one and are nonnegative for $0 \leq a, b \leq 1$. This gives a continuous, piecewise bilinear estimate used when sampling image intensities at subpixel positions. It does not create new measured image detail.

Numerical example (illustrative intensities)

Neighbor	Location	Intensity	Weight
I_{00}	(10,20)	100	0.375
I_{10}	(11,20)	120	0.125
I_{01}	(10,21)	140	0.375
I_{11}	(11,21)	160	0.125

At $(x, y) = (10.25, 20.5)$, $a = 0.25$ and $b = 0.5$. Interpolate the top row: $T = 0.75 \cdot 100 + 0.25 \cdot 120 = 105$. Interpolate the bottom row: $B = 0.75 \cdot 140 + 0.25 \cdot 160 = 145$. Interpolate vertically: $I = 0.5 \cdot 105 + 0.5 \cdot 145 = 125$. The expanded formula gives $37.5 + 15 + 52.5 + 20 = 125$.

Two-frame tracking validation setup

For each saved feature p_A , Lucas-Kanade produces $p_{LK} = p_A + d$. An independent patch search measures a comparison location q_B from frame B pixels. The discrepancy is $\|p_{LK} - q_B\|$. This cross-check uses a different estimation method; it is not manually annotated ground truth. The comparison locations are integer-valued, so errors below about one pixel cannot establish accurate subpixel motion.

5. Consecutive-frame tracking validation: video1

Frame pair: 1770 -> 1771, zero-based. Timestamps: 29.815720 s -> 29.832565 s. The current script samples approximately once per second within the first 30 seconds and selects the largest mean absolute grayscale difference. It detects and successfully tracks 100 points; a successful status is not a guarantee of accuracy.

Independent measurement: search each 15x15 grayscale patch in frame B within +/-45 pixels of its frame A position using TM_CCOEFF_NORMED [6]. The search never uses the LK destination. Require patch standard deviation ≥ 5 , correlation ≥ 0.95 , and a best-peak advantage ≥ 0.05 over matches more than 3 pixels from the best peak. Uncertain matches are excluded from reported high-confidence means.



Figure 3. Frame A (left): yellow initial points and CSV IDs. Frame B (right): green independently matched locations and magenta LK predictions. Overlap reflects agreement at display scale.

ID	A (x,y)	LK in B (x,y)	Pixel match (x,y)	NCC	Error px
1	329,141	328.834,140.806	329,141	0.9876	0.255
2	391,34	390.945,33.893	391,34	0.9939	0.120
3	210,179	209.981,178.839	210,179	0.9968	0.162

Worked actual-image comparison, point 1: $dx=328.834076-329=-0.165924$; $dy=140.806015-141=-0.193985$. Magnitude=0.255266 pixels. Independent frame B pixel location is (329,141); prediction error= $\sqrt{(328.834076-329)^2+(140.806015-141)^2}=0.255266$ pixels.

High-confidence independent matches: 97; mean discrepancy=0.160 pixels; median=0.162 pixels. LK median magnitude over all 100 tracks=0.161 pixels. Static background corners dominate these feature sets. Small displacements do not demonstrate object tracking on the moving object.

5. Consecutive-frame tracking validation: video2

Frame pair: 450 -> 451, zero-based. Timestamps: 14.944434 s -> 14.977644 s. The current script samples approximately once per second within the first 30 seconds and selects the largest mean absolute grayscale difference. It detects and successfully tracks 100 points; a successful status is not a guarantee of accuracy.

Independent measurement: search each 15x15 grayscale patch in frame B within +/-45 pixels of its frame A position using TM_CCOEFF_NORMED [6]. The search never uses the LK destination. Require patch standard deviation ≥ 5 , correlation ≥ 0.95 , and a best-peak advantage ≥ 0.05 over matches more than 3 pixels from the best peak. Uncertain matches are excluded from reported high-confidence means.



Figure 4. Frame A (left): yellow initial points and CSV IDs. Frame B (right): green independently matched locations and magenta LK predictions. Overlap reflects agreement at display scale.

ID	A (x,y)	LK in B (x,y)	Pixel match (x,y)	NCC	Error px
1	239,145	239.520,144.997	240,145	0.9685	0.480
3	616,79	615.998,79.000	616,79	1.0000	0.002
5	311,234	304.570,270.306	311,234	0.9963	36.872

Worked actual-image comparison, point 1: $dx=239.519806-239=0.519806$; $dy=144.997208-145=-0.002792$. Magnitude= 0.519813 pixels. Independent frame B pixel location is (240,145); prediction error= $\sqrt{(239.519806-240)^2+(144.997208-145)^2}=0.480202$ pixels.

High-confidence independent matches: 93; mean discrepancy= 0.560 pixels; median= 0.091 pixels. LK median magnitude over all 100 tracks= 0.099 pixels. Static background corners dominate these feature sets. Small displacements do not demonstrate object tracking on the moving object.

6. Tracking failure analysis and limitations

Video2 point 5 illustrates why LK status alone is insufficient. It begins at (311,234) on the door handle. LK predicts (304.570,270.306), a magnitude of 36.872 pixels. Independent patch matching instead finds (311,234) with correlation 0.9963. The methods disagree by 36.872 pixels. The unchanged door-handle patch supports interpreting this track as a likely mismatch, not evidence of fast physical movement.



Figure 5. Enlarged identical-coordinate crops around the door handle: initial feature (yellow), independently matched location (green), and LK prediction (magenta).

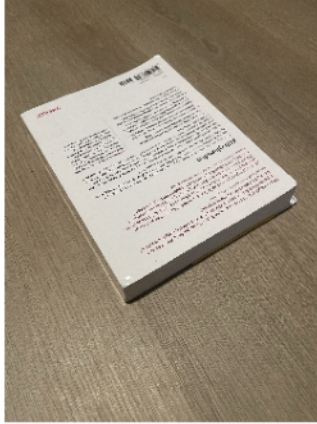
Independent patch matching can also fail on repeated textures, illumination changes, occlusion, or nonrigid motion. Its confidence rules reduce ambiguity but do not establish ground truth. No manually clicked tracking measurements were available. Reported errors are disagreements between two estimators and include integer reference quantization. A manually annotated feature correspondence would provide a stronger check if required by the instructor.

Video1 has mean frame difference 0.793/255 at the selected pair, while video2 has 6.059/255. This can reflect brightness variation as well as motion. Because the tracked points are mostly static background features, these pairs validate small displacement estimates and expose mismatches rather than demonstrating robust moving-object trajectories. No cross-video physical speed comparison is inferred.

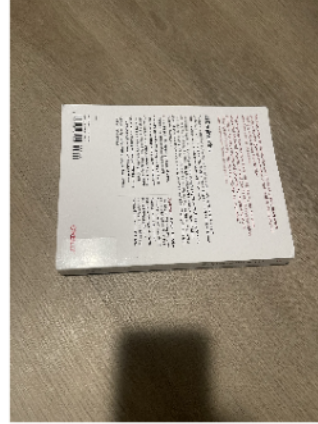
7. Planar object, four viewpoints, and camera data

The book back cover is treated as a plane. Only its four cover corners and three interior printed details are used. The spine, thickness, and protruding pages are excluded. View1 defines the projective reference coordinates. The photographs are shown below with their orientation as decoded by OpenCV.

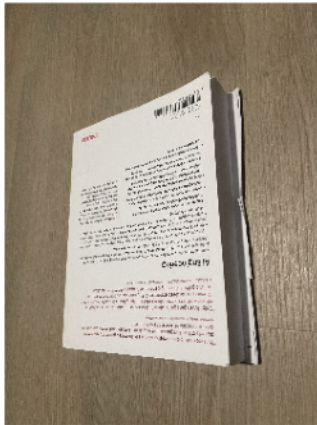
view1



view2



view3



view4

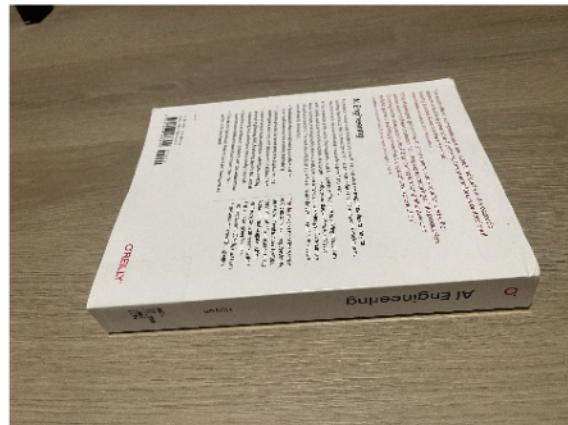


Figure 6. Four source photographs. Relative positions are qualitative observations; distances and angles were not measured.

View	Relative viewpoint	Decoded size (w x h)
view1	Oblique overhead	3024 x 4032
view2	Closer to overhead	3024 x 4032
view3	Rotated oblique	3024 x 4032
view4	Lower grazing angle	4032 x 3024

Camera metadata retained in all four PNGs: Apple iPhone 12 Pro Max; lens 'iPhone 12 Pro Max back triple camera 5.1mm f/1.6'; focal length 5.1 mm; 35 mm equivalent 25 mm; aperture f/1.6; exposure 1/60 s; ISO 200. Views 1-3 have EXIF orientation 6; view4 has orientation 1. Stored raster size is 4032 x 3024; the table gives oriented OpenCV coordinates used by the CSVs.

8. Camera limitations and planar homography derivation

EXIF gives focal length in millimeters, not a calibrated focal length in pixels. Principal point, pixel focal lengths, distortion coefficients, camera-to-object distance, and metric poses were not measured. No calibrated intrinsic matrix K or metric camera position is claimed. Relative viewpoints and source images document the acquisition; exact camera positions remain unavailable. This is a planar projective boundary reconstruction, not metric 3D structure recovery [4].

$$s[x', y', 1]^T = H[x, y, 1]^T$$

Write the rows of H as $h_{11}, h_{12}, h_{13}; h_{21}, h_{22}, h_{23}; h_{31}, h_{32}, h_{33}$. Fix $h_{33}=1$ for these fitted matrices, removing arbitrary scale. Multiplying H by the source point gives $q=(X,Y,W)$; divide by W to obtain image coordinates.

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + 1}$$

$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + 1}$$

Cross-multiply and rearrange each correspondence to obtain two linear equations:

$$xh_{11} + yh_{12} + h_{13} - xx'h_{31} - yy'h_{32} = x'$$

$$xh_{21} + yh_{22} + h_{23} - xy'h_{31} - yy'h_{32} = y'$$

With $h=[h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}]^T$, the two rows in the 8×8 system $Mh=c$ are:

$$\begin{aligned} [x, y, 1, 0, 0, 0, -x*x', -y*x'] &\rightarrow x' \\ [0, 0, 0, x, y, 1, -x*y', -y*y'] &\rightarrow y' \end{aligned}$$

Stack four nondegenerate corner correspondences, solve for h , and reshape into H . The actual implementation uses `cv2.findHomography(source_corners, reference_corners, method=0)`, which handles the numerical estimation. No validation points enter this calculation. `cv2.perspectiveTransform` applies the resulting H [4].

Numerical row example, view2 corner 1: `source=(2460.098389,1017.304626)`, `reference=(2788.524658,1736.861572)`. The x -equation includes $-x*x'=-6860045.018417$ and $-y*x'=-2836779.035801$, with right-hand side $x'=2788.524658$. The remaining seven rows follow the same construction.

9. Corner correspondences and fitted matrices

View	Corner	Source x	Source y
view1	1	2788.525	1736.862
view1	2	1127.803	2735.557
view1	3	142.525	1085.538
view1	4	1462.426	514.855
view2	1	2460.098	1017.305
view2	2	2726.557	2444.012
view2	3	452.361	2580.480
view2	4	501.934	1004.898
view3	1	2131.672	3455.114
view3	2	545.311	3262.818
view3	3	650.656	1253.021
view3	4	1933.377	657.526
view4	1	3157.702	525.711
view4	2	3776.221	1916.751
view4	3	553.412	2167.975
view4	4	1027.765	739.717

Corner IDs retain the same physical order across rotations of the book. The reference coordinates are the four view1 rows above. Matrices below map each named source view into view1; entries are printed with sufficient precision for the worked example.

```
view2 -> view1
[[ 2.0892316318e-01 -7.8533259012e-01 2.0755966909e+03]
 [ 3.1694797258e-01 3.9312864117e-01 -6.4332461071e+01]
 [-1.5809889323e-04 3.0558114107e-05 1.0000000000e+00]]
```

```
view3 -> view1
[[ 2.2393821206e+00 5.9976230058e-01 -2.0365912893e+03]
 [ 5.4618843567e-02 7.2869547675e-01 3.6135737166e+02]
 [ 4.5799331825e-04 -7.2825919575e-05 1.0000000000e+00]]
```

```
view4 -> view1
[[ 2.4417852437e-01 -8.7067051519e-01 1.9583242465e+03]
 [ 4.9359323362e-01 8.7592416263e-01 -6.0418464667e+02]
 [-9.6947898470e-05 2.2973394690e-04 1.0000000000e+00]]
```

10. Independent planar validation: worked calculation

Three additional physical details on the flat cover were manually selected in each view. The same validation point IDs identify the same details in every image. These points are excluded from homography estimation. Predicted locations are compared with the actual manually clicked reference locations in view1.

$$e_i = \sqrt{(x_{pred,i} - x_{actual,i})^2 + (y_{pred,i} - y_{actual,i})^2}$$

View2 validation point 1: $p=[694.032776, 1104.147705, 1]^T$.

Using the full-precision H printed on the previous page:

$X=0.2089231632*694.032776+(-0.7853325901)*1104.147705+2075.5966909176=1353.473037$

$Y=0.3169479726*694.032776+0.3931286412*1104.147705+(-64.3324610710)=589.711907$

$W=-0.0001580989*694.032776+0.0000305581*1104.147705+1=0.924014858$

Divide by W: predicted=(1464.774105, 638.206087).

Actual manually selected view1 location=(1481.016357, 657.526184).

Residual=(-16.242252, -19.320097).

$e=\sqrt{(-16.242252)^2 + (-19.320097)^2}=25.240382$ pixels.

The saved float32 perspectiveTransform result rounds to 25.240396 pixels.

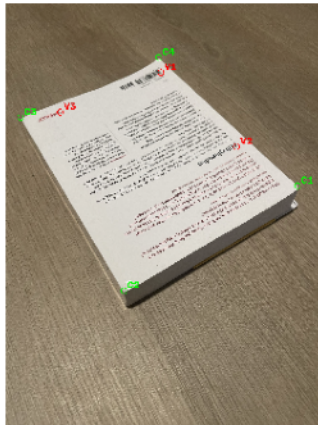
View	Corner fitting error (px)	Independent error (px)
view2	0.000	25.139
view3	0.000	26.024
view4	0.000	29.803
Overall	0.000	26.989

Corner fitting errors are zero at the saved precision because the same four corners define H. They verify fitting consistency, not independent accuracy. Independent errors measure agreement at three held-out, manually identified points per source view. The sample is small; no universal accuracy threshold or metric 3D accuracy is inferred.

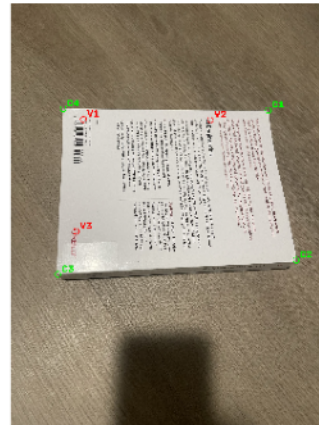
11. Planar validation correspondences and evidence

View	ID	Source (x,y)	Predicted (x,y)	Actual view1 (x,y)	Error px
view2	1	694.0,1104.1	1464.8,638.2	1481.0,657.5	25.240
view2	2	1908.6,1110.4	2188.5,1334.5	2206.0,1364.7	34.921
view2	3	625.9,2171.1	518.2,1020.8	520.5,1035.9	15.254
view3	1	1865.2,924.3	1508.0,636.1	1481.0,657.5	34.415
view3	2	1927.2,2499.8	2221.8,1345.6	2206.0,1364.7	24.800
view3	3	879.9,1277.8	534.6,1023.4	520.5,1035.9	18.858
view4	1	1204.5,795.5	1463.2,644.6	1481.0,657.5	21.976
view4	2	2497.3,674.6	2169.8,1335.8	2206.0,1364.7	46.321
view4	3	869.6,1730.7	505.5,1021.1	520.5,1035.9	21.111

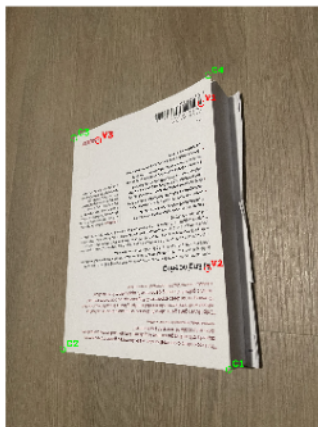
view1



view2



view3



view4

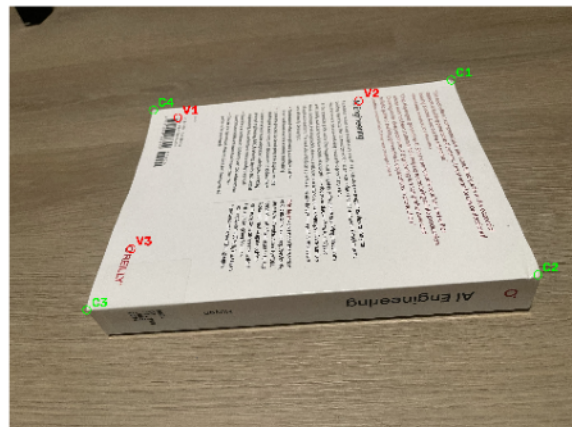


Figure 7. Fitting corners C1-C4 (green) and independent validation details V1-V3 (red). Reference validation locations come from the saved actual_x/actual_y columns. Identical labels designate physical correspondences.

Exact validation source/reference coordinates and errors are included in results/sfm_validation_results.csv. The table here rounds coordinates for readability; calculations use unrounded saved values.

12. Reconstructed boundary and interpretation

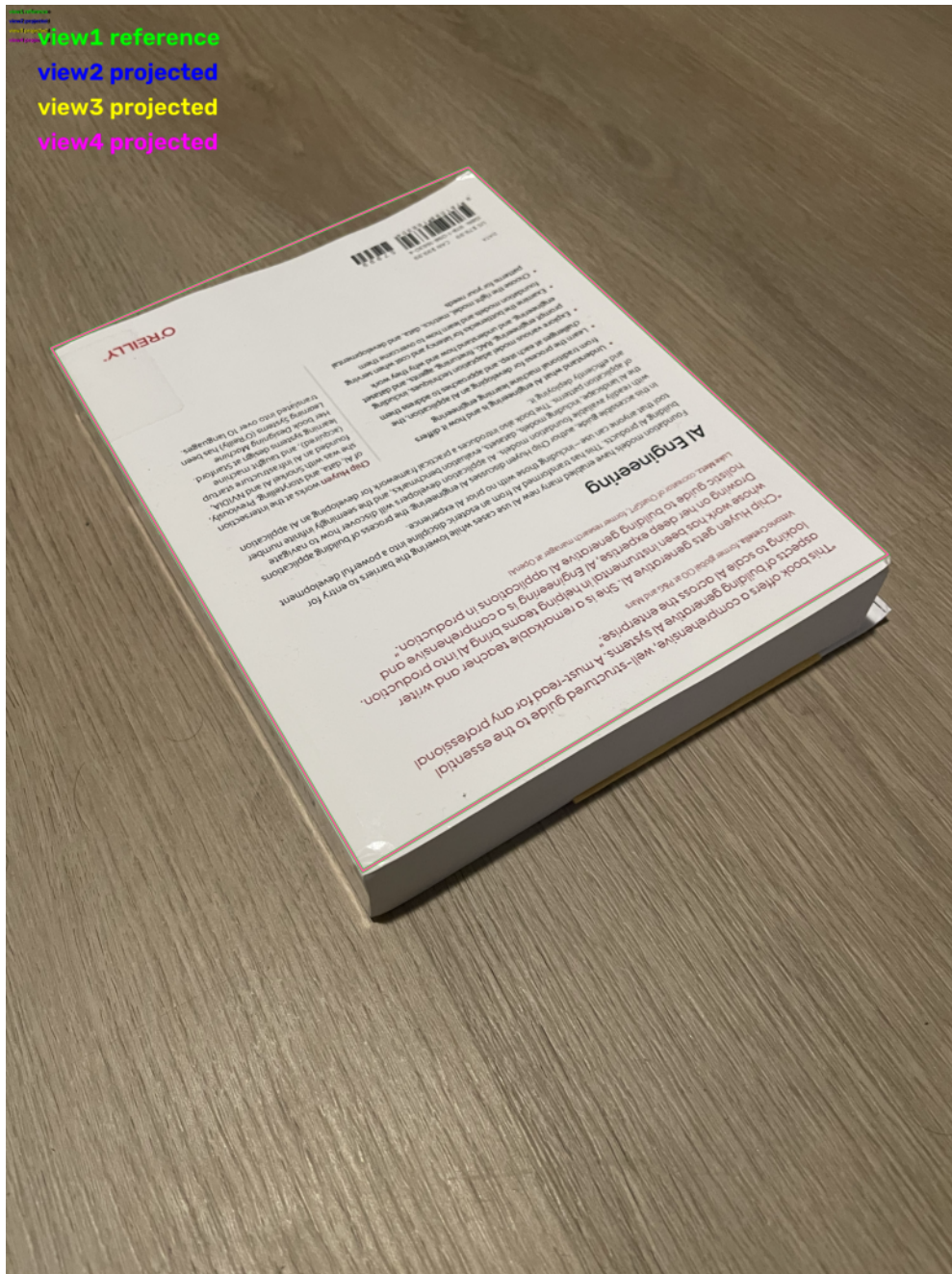


Figure 8. View1 reference boundary and projected boundaries from views 2, 3, and 4. Colored labels and nested line widths distinguish nearly coincident outlines. This reconstructs a planar boundary in view1 pixel coordinates, not in physical units.

The independent mean error is 26.989 pixels, about 0.535% of the reference image diagonal. Individual errors range from 15.254 to 46.321 pixels. Reporting relative scale provides context; it does not establish that the accuracy is acceptable for an unspecified application.

Possible contributors include manual point uncertainty, imperfect planarity, image resampling, and lens distortion. The experiment does not isolate their contributions. Boundary coincidence is expected because its endpoints were used to fit H. The held-out interior correspondences provide the independent accuracy evidence. Physical 3D reconstruction would require additional geometric constraints and appropriate camera information.

13. Conclusions, references, and submission

Dense Farneback flow supplies apparent image-motion direction and magnitude. Consecutive-frame Lucas-Kanade tracking estimates subpixel displacement; independent image-patch comparisons show agreement on many background features and identify a likely erroneous large track. The selected pairs provide limited evidence of moving-object tracking. Four-view homographies map a planar book boundary into view1; held-out manual points give an independent mean discrepancy of 26.989 pixels. No metric 3D structure or calibrated camera poses are claimed.

[1] OpenCV. Optical Flow. https://docs.opencv.org/4.x/d4/dee/tutorial_optical_flow.html

[2] G. Farneback. Two-Frame Motion Estimation Based on Polynomial Expansion. SCIA 2003, pp. 363-370. DOI: 10.1007/3-540-45103-X_50.

[3] B. D. Lucas and T. Kanade. An Iterative Image Registration Technique with an Application to Stereo Vision. IJCAI 1981, pp. 674-679. <https://publications.ri.cmu.edu/an-iterative-image-registration-technique-with-an-application-to-stereo-vision-ijcai>

[4] OpenCV. Basic concepts of the homography explained with code. https://docs.opencv.org/4.x/d9/dab/tutorial_homography.html

[5] OpenCV. Geometric Image Transformations (INTER_LINEAR: bilinear interpolation). https://docs.opencv.org/4.x/da/d54/group_imgproc_transform.html

[6] OpenCV. Template Matching. https://docs.opencv.org/4.x/d4/dc6/tutorial_py_template_matching.html

Course-provided supplementary resource: First Principles of Computer Vision, Optical Flow | Structure from Motion | Object Tracking playlist. <https://www.youtube.com/playlist?list=PL2zRqk16wsdoYzrWStffqBAoUY8XdvatV> . Listed as a course resource; no specific unverified lecture content is attributed to it.

Submission checklist

Add the accessible GitHub URL on page 1. Upload the repository code, README, source images, and relevant outputs. Submit this PDF and a screen-recording or properly captured working-system demonstration in Classroom. Generated flow MP4s show results, but are not by themselves proof of the complete interactive workflow. Camera positions are qualitative; if measured distances/angles are available, add them. The mathematical workouts here are typed; no photographs of handwritten worksheets are used.